

RV32I A* Accelerator SoC

2026-06-23 개발 보고서

Day 1 - CPU instruction fetch 및 RV32I subset 싱글사이클 skeleton

1. 프로젝트 개요

항목	내용
프로젝트명	RV32I A* Accelerator SoC
목표	RV32I 기반 CPU를 직접 구현하고, 이후 MMIO bus와 A* pathfinding accelerator를 통합하는 FPGA SoC 프로젝트
타겟 보드	Terasic DE2-115 / Intel Cyclone IV
언어	Verilog HDL
현재 단계	CPU instruction fetch, register file, immediate generation, ALU, writeback 검증

2. 오늘 한 일 요약

- RV32I subset instruction 범위를 1차로 정했다.
- opcode/funct3/funct7 기반 decode 방향을 확인했다.
- imem, reg_file, imm_gen, alu, cpu_core를 모듈 단위로 나눠 작성했다.
- program.hex를 만들어 instruction memory 초기값으로 사용했다.
- tb_cpu_core.v testbench를 작성하고 Vivado xsim으로 register writeback을 검증했다.
- Questa FSE는 라이선스 문제로 실행이 막혀, 당일 검증은 Vivado xsim으로 우회했다.

3. RV32I Subset 및 Decode 결정

구분	Instruction
R-type	ADD, SUB, AND, OR, XOR
I-type	ADDI, SLLI, XORI, ORI, ANDI
U-type	LUI
Memory	LW, SW
Branch	BEQ, BNE, BLT, BGE
Jump	JAL, JALR

오늘 실제 검증 범위는 ADDI와 R-type ADD/SUB/AND/OR/XOR로 제한했다. LW/SW, branch, jump는 아직 전체 datapath에 연결되지 않았다.

```
wire [6:0] OP      = IR[6:0];
wire [4:0] RD      = IR[11:7];
wire [2:0] funct3  = IR[14:12];
wire [4:0] RS1     = IR[19:15];
wire [4:0] RS2     = IR[24:20];
wire [6:0] funct7  = IR[31:25];
```

4. 모듈별 구현 내용

파일	역할	설계 의도
cpu_core.v	PC, instruction fetch, field decode, reg_file/imm_gen/alu 연결, writeback 제어	control unit을 아직 분리하지 않고 top에서 데이터 흐름을 눈에 보이게 연결했다.
imem.v	\$readmemh로 program.hex를 읽는 instruction memory	PC byte address에서 하위 2비트를 버리고 addr[11:2]로 word index를 만들었다.
reg_file.v	32개 32-bit register, rs1/rs2 조합 read, rd 동기 write	x0 write를 막고 x0 read를 항상 0으로 보유했다.
imm_gen.v	I-type과 U-type immediate를 32-bit로 생성	오늘 필요한 ADDI 계열과 LUI를 먼저 지원했다.
alu.v	opcode/funct3/funct7 기반 산술/논리 연산	초기 학습 단계라 ALU 안에서 instruction 종류를 직접 구분했다.
program.hex	테스트용 RISC-V machine code	ADDI로 값을 만들고 R-type 연산 결과가 register에 writeback되는지 확인했다.
tb_cpu_core.v	clock/reset 생성, register 값 자동 검사	hierarchical reference로 내부 regfile 값을 읽어 PASS/FAIL을 바로 확인했다.

5. 핵심 코드 설명

IMEM은 program.hex 파일의 32-bit instruction 값을 메모리 배열에 채운 뒤, PC 주소에 해당하는 instruction을 출력한다. DEPTH가 1024이므로 10-bit word index인 addr[11:2]를 사용했다.

```
reg [31:0] mem [0:DEPTH-1];

initial begin
    $readmemh(INIT_FILE, mem);
end

assign instr = mem[addr[11:2]];
```

reg_file은 RISC-V 규칙에 맞게 x0을 보호한다. W_WA가 0이 아닐 때만 write하고, read할 때도 x0은 0으로 고정한다.

```
else if (W_WEN && (W_WA != 5'd0)) begin
    regs[W_WA] <= W_WD;
end

assign scr1 = (RS1 == 5'd0) ? 32'd0 : regs[RS1];
assign scr2 = (RS2 == 5'd0) ? 32'd0 : regs[RS2];
```

6. 검증 결과

Vivado xsim에서 tb_cpu_core를 top으로 두고 실행했으며, 모든 register check가 통과했다.

```
OK: x1 = 5 (0x00000005)
OK: x2 = 7 (0x00000007)
OK: x3 = 12 (0x0000000c)
OK: x4 = 7 (0x00000007)
OK: x5 = 4 (0x00000004)
OK: x6 = 7 (0x00000007)
OK: x7 = 2 (0x00000002)
PASS: basic ADDI/R-type register writeback test
```

또한 Questa의 vlog 컴파일러로 최신 RTL/testbench 문법 컴파일을 확인했다.

```
vlog cpu_core.v imem.v reg_file.v imm_gen.v alu.v tb_cpu_core.v
Errors: 0, Warnings: 0
```

7. 헛갈렸던 점과 해결

- Quartus는 합성/배치/보드 업로드용이고, testbench 실행은 simulator가 담당한다는 점을 정리했다.
- \$readmemh의 상대경로 기준은 Verilog 파일 위치가 아니라 simulator 실행 위치라는 점을 확인했다.
- input은 보통 wire로 볼 수 있고, output은 assign이면 wire, always block에서 대입하면 reg로 선언한다는 점을 정리했다.
- reg_file은 instruction 전체를 받지 않고, cpu_core에서 instruction field를 잘라 주소와 제어신호만 넘기는 구조로 정했다.

8. 현재 한계

현재 구현은 CPU 전체가 아니라 초기 실행 경로 검증이다. 아래 항목은 아직 구현 전이거나 미완성이다.

- 5-stage pipeline register, forwarding, load-use stall, branch flush
- data memory 기반 LW/SW
- branch/jump PC redirect
- MMIO bus, A* accelerator
- interrupt/CSR, FPGA board bring-up

9. 다음 작업 계획

- Lui, Andi, Ori, Xori, Slli test vector를 program.hex에 추가한다.
- imm_gen.v에 S-type, B-type, J-type immediate를 추가한다.
- dmem.v를 만들고 LW/SW 경로를 구현한다.
- cpu_core.v의 PC update를 pc_next 구조로 바꾼다.
- BEQ/BNE/BLT/BGE와 JAL/JALR을 실제 PC redirect와 연결한다.
- 코드가 커지면 decoder.v 또는 control_unit.v로 decode/control 신호를 분리한다.

10. Day 1 결론

Day 1에는 RV32I CPU 전체 기능을 만든 것이 아니라, instruction fetch부터 ALU writeback까지 이어지는 가장 작은 CPU 실행 경로를 만들고 검증했다. 이 골격 위에 memory, branch/jump, pipeline, MMIO, accelerator를 단계적으로 붙이면 된다.